

Express Routing e API REST

1. Che cos'è una rotta

Il **routing** è il processo con cui il server decide quale operazione eseguire in base a una richiesta HTTP.

In Express, una rotta è composta da tre elementi:

```
app.get('/posts', (req, res) => {  
  res.send('Lista dei post');  
});
```

- **metodo HTTP:** `get`
- **percorso (path):** `/posts`
- **callback:** la funzione che riceve `req` e `res` ed esegue la logica

`req` rappresenta la richiesta del client; `res` serve a costruire e inviare la risposta. Ogni rotta deve concludersi inviando una risposta, per esempio con `res.send()` o `res.json()`.

2. Risorse, CRUD e convenzioni REST

Una API REST organizza gli endpoint intorno a delle **risorse**. In questo esercizio la risorsa è `posts`.

Le operazioni CRUD sono:

Operazione	Metodo HTTP	Endpoint	Significato
Index	GET	/posts	Legge tutti i post
Show	GET	/posts/:id	Legge un singolo post
Create	POST	/posts	Crea un nuovo post
Update	PUT	/posts/:id	Aggiorna un post
Delete	DELETE	/posts/:id	Elimina un post

Il metodo HTTP esprime l'operazione, quindi non servono endpoint come `/getPost` o `/deletePost/1`.

REST è una convenzione architetturale: Express non decide la logica al posto nostro. Siamo noi a collegare correttamente metodi, endpoint e operazioni.

3. Parametri dinamici

I due punti definiscono un segmento dinamico dell'URL:

```
router.get('/:id', (req, res) => {  
  const id = req.params.id;
```

```
res.send(`Dettaglio del post ${id}`);  
});
```

Con una richiesta a `/posts/3`, `req.params.id` contiene la stringa `'3'`.

Se dobbiamo confrontare l'ID con un numero, dobbiamo convertirlo:

```
const id = Number(req.params.id);
```

4. Separare le rotte con Router

`express.Router()` crea un router dedicato a una risorsa. In questo modo `server.js` rimane leggero e le rotte dei post sono raccolte nello stesso file.

Struttura essenziale:

```
express-blog-routing/  
├── server.js  
├── data/  
│   └── posts.js  
└── routers/  
    └── posts.js
```

routers/posts.js

```
import express from 'express';  
  
const router = express.Router();  
  
router.get('/', (req, res) => {  
  res.send('Lista dei post');  
});  
  
router.get('/:id', (req, res) => {  
  res.send(`Dettaglio del post ${req.params.id}`);  
});  
  
router.post('/', (req, res) => {  
  res.send('Creazione di un nuovo post');  
});  
  
router.put('/:id', (req, res) => {  
  res.send(`Aggiornamento del post ${req.params.id}`);  
});  
  
router.delete('/:id', (req, res) => {  
  res.send(`Cancellazione del post ${req.params.id}`);  
});
```

```
});  
  
export default router;
```

Nel router scriviamo `/` e `/:id`, perché il prefisso `/posts` verrà aggiunto quando registriamo il router nell'applicazione. (Vedi `server.js`, riga `app.use('/posts', postsRouter);`)

server.js

```
import express from 'express';  
import postsRouter from './routers/posts.js';  
  
const app = express();  
const PORT = 3000;  
  
app.use('/posts', postsRouter);  
  
app.listen(PORT, () => {  
  console.log(`Server avviato su http://localhost:${PORT}`);  
});
```

Express combina il prefisso e il path dichiarato nel router:

```
/posts + /      -> /posts  
/posts + /:id  -> /posts/:id
```

5. Bonus: risposte JSON

Possiamo salvare l'array in `data/posts.js`, esportarlo e importarlo nel router:

```
// data/posts.js  
const posts = [  
  { id: 1, title: 'Primo post' },  
  { id: 2, title: 'Secondo post' },  
];  
  
export default posts;
```

Index in JSON

```
import posts from '../data/posts.js';  
  
router.get('/', (req, res) => {
```

```
    res.json(posts);  
  });
```

Show in JSON

```
router.get('/:id', (req, res) => {  
  const id = Number(req.params.id);  
  const post = posts.find((post) => post.id === id);  
  
  if (!post) {  
    return res.status(404).json({ error: 'Post non trovato' });  
  }  
  
  res.json(post);  
});
```

`Array.prototype.find()` restituisce il primo elemento che soddisfa la condizione oppure `undefined`. Se il post non esiste, rispondiamo con lo status HTTP `404 Not Found`.

6. Promemoria rapido

- Usa nomi plurali per le risorse: `/posts`.
- Usa il metodo HTTP per descrivere l'operazione CRUD.
- Non inserire l'ID nella rotta `POST /posts`: prima della creazione potrebbe non esistere.
- Ricorda che `req.params.id` è una stringa.
- Usa `res.send()` per inviare testo e `res.json()` per inviare JSON.
- Monta il router con `app.use('/posts', postsRouter)`.
- Prova ogni rotta con Postman o con un file `.http`, scegliendo il metodo HTTP corretto.